PRACTICAL 1(a): Hello World Android Application

Aim: Develop a simple Hello World Android application demonstrating project creation, the Android Studio interface, Android components, AVD creation, and application execution.

Project Name: HelloWorld
Template: Empty Views Activity
Language: Java

Steps

1. Open Android Studio.
2. Click **New Project**.
3. Select **Empty Views Activity**.
4. Enter the project name HelloWorld.
5. Select **Java** as the language.
6. Click **Finish** and wait for Gradle to finish syncing.
7. Open MainActivity.java and enter the Java code below.
8. Open activity_main.xml and enter the XML code.
9. Check the AndroidManifest.xml and resource files.
10. Open **Device Manager** and create an Android Virtual Device (AVD) if you don't already have one.
11. Start the emulator.
12. Click **Run** ▶ to execute the application.

PRACTICAL 1(b): Android Programming Resources

Aim: Develop an Android application demonstrating programming resources such as colors.xml, themes.xml, strings.xml, dimens.xml and drawable resources, and apply them to UI components.

Project Name: Practical2R
Template: Empty Views Activity
Language: Java

Steps

1. Open Android Studio.
2. Create a new project using **Empty Views Activity**.
3. Enter the project name Practical2R.
4. Select Java as the programming language.
5. Create the required Java activity files.
6. Create the corresponding XML layout files.
7. Add the color, string, dimension and theme resources.
8. Add an image named pebbles to the res/drawable folder.
9. Register the activities in AndroidManifest.xml.
10. Run the application.
11. Click the buttons to view the Theme Demo, String Demo, Drawable Demo and Dimension Demo screens.

PRACTICAL 1(c): Android Activity Lifecycle Using Logcat

Aim: Develop an Android application demonstrating complete Activity Lifecycle methods and analyse execution using Logcat.

Project Name: LogCatDemo
Template: Empty Views Activity
Language: Java

Steps

1. Open Android Studio.
2. Create a new project using **Empty Views Activity**.
3. Enter the project name LogCatDemo.
4. Select Java as the programming language.
5. Open MainActivity.java.
6. Write the Java code below to implement the Activity Lifecycle methods.
7. Open activity_main.xml and add the TextView.
8. Run the application on an Android emulator.
9. Open **Logcat** in Android Studio.

10. Filter the Logcat messages using the tags on start invoked, on resume invoked, on pause invoked, on stop invoked or on destroy invoked.
11. Minimize the application, reopen it and close it to observe the lifecycle messages.

PRACTICAL 2(a): Create and Integrate a Fragment and Demonstrate Fragment Lifecycle

Aim: Develop an Android application to create and integrate a Fragment and demonstrate Fragment lifecycle methods.

Project Name: FragmentLifecycleDemo
Fragment Name: DemoFragment
Fragment Layout: fragment_demo
Language: Java

Steps

1. Open Android Studio.
2. Create a new project using **Empty Views Activity**.
3. Set the project name to FragmentLifecycleDemo and select Java.
4. In the Java package, select **New → Fragment → Fragment (Blank)**.
5. Name the fragment DemoFragment.
6. Name the fragment layout fragment_demo.
7. Open DemoFragment.java and enter the code below.
8. Open activity_main.xml and add the fragment container.
9. Open MainActivity.java and add the code to load the fragment.
10. Open fragment_demo.xml and add the TextView.
11. Run the application.
12. Open **Logcat** in Android Studio to observe the Fragment lifecycle messages.

PRACTICAL 2(b): Fragment-Based UI Navigation Between Multiple Screens

Aim: Develop an Android application implementing Fragment-based UI navigation between multiple screens.

Project Name: Fragment Lifecycle
Template: Empty Views Activity
Language: Java

Files required:

1. MainActivity.java
2. FragmentOne.java
3. FragmentTwo.java
4. activity_main.xml
5. fragment_one.xml
6. fragment_two.xml

Steps

1. Open Android Studio.
2. Create an **Empty Views Activity** project.
3. Name the project FragmentLifecycle.
4. Select Java as the language.
5. Create FragmentOne.java.
6. Create FragmentTwo.java.
7. Create the layouts fragment_one.xml and fragment_two.xml.
8. Add a button and a FrameLayout to activity_main.xml.
9. Write the code in MainActivity.java to switch between the fragments.
10. Run the application.
11. Click **Change Fragment** to switch between Fragment One and Fragment Two.

PRACTICAL 2(c): Communication Between Activity and Fragment

Aim: Develop an Android application demonstrating communication between Activity and Fragment.

Project Name: ActivityFragmentCommunication
Fragment Name: DemoFragment
Fragment Layout: fragment_demo
Language: Java

Steps

1. Open Android Studio.
2. Create a new project using **Empty Views Activity**.
3. Enter the project name ActivityFragmentCommunication.

4. Select Java as the programming language.
5. Create a blank Fragment named DemoFragment.
6. Name the Fragment layout fragment_demo.
7. Open activity_main.xml and create a FrameLayout to hold the Fragment.
8. Open fragment_demo.xml and add a TextView and Button.
9. Open DemoFragment.java and write the code for sending and receiving messages.
10. Open MainActivity.java and implement the FragmentCommunication interface.
11. Run the application.
12. The Fragment will display a message from the Activity.
13. Click **Send Message to Activity** to send a message back to the Activity.

PRACTICAL 3(a): LinearLayout — Horizontal and Vertical

Aim: Develop an Android application using LinearLayout (Horizontal and Vertical) to design a structured user interface.

Project Name: LinearLayout
Language: Java
Main File: activity_main.xml

Steps

1. Open Android Studio.
2. Create a new project using **Empty Views Activity**.
3. Enter the project name LinearLayout.
4. Select Java as the language.
5. Open activity_main.xml.
6. Replace the existing XML with the code below.
7. Run the application on an emulator.

PRACTICAL 3(a): LinearLayout — Horizontal and Vertical

Aim: Develop an Android application using LinearLayout (Horizontal and Vertical) to design a structured user interface.

Project Name: LinearLayout
Language: Java
Main File: activity_main.xml

Steps

1. Open Android Studio.
2. Create a new project using **Empty Views Activity**.
3. Enter the project name LinearLayout.
4. Select Java as the language.
5. Open activity_main.xml.
6. Replace the existing XML with the code below.
7. Run the application on an emulator.

Code: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:id="@+id/main"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical"
   android:padding="20dp">

   <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Student Registration"
      android:textSize="26sp"
      android:textStyle="bold"
      android:layout_gravity="center"
      android:layout_marginBottom="20dp"/>

   <EditText
```

```
    android:id="@+id/etName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Name"
    android:layout_marginBottom="10dp"/>

<EditText
    android:id="@+id/etEmail"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Email"
    android:inputType="textEmailAddress"
    android:layout_marginBottom="20dp"/>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center">

    <Button
        android:id="@+id/btnSubmit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Submit"/>

    <Button
        android:id="@+id/btnReset"
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Reset"
        android:layout_marginStart="20dp"/>

    </LinearLayout>

</LinearLayout>
```

Code: MainActivity.java

```
package com.example.linearlayout;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    EditText etName, etEmail;
    Button btnSubmit, btnReset;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
```

```
        etName = findViewById(R.id.etName);
        etEmail = findViewById(R.id.etEmail);
        btnSubmit = findViewById(R.id.btnSubmit);
        btnReset = findViewById(R.id.btnReset);

        btnSubmit.setOnClickListener(v -> {
            String name = etName.getText().toString();
            String email = etEmail.getText().toString();

            Toast.makeText(
                    this,
                    "Name: " + name + "\nEmail: " + email,
                    Toast.LENGTH_LONG
            ).show();
        });

        btnReset.setOnClickListener(v -> {
            etName.setText("");
            etEmail.setText("");
        });
    }
}
```

Expected output

A student registration form containing:

- Student Registration heading.
- Name input field.
- Email input field.
- Submit button.

- Reset button.

The fields are arranged vertically, while the buttons are arranged horizontally.

PRACTICAL 3(b): RelativeLayout

Aim: Develop an Android application using RelativeLayout to align UI components dynamically.

Project Name: RelativeLayout
Main File: activity_main.xml

Steps

1. Open Android Studio.
2. Create an **Empty Views Activity** project.
3. Name the project RelativeLayout.
4. Open activity_main.xml.
5. Replace the existing XML with the code below.
6. Add an image named zoro to the res/drawable folder.
7. Run the application.

PRACTICAL 3(c): TableLayout — Login/Registration Form

Aim: Develop an Android application using TableLayout to design a structured login or registration form.

Project Name: TableLayout
Main File: activity_main.xml

Steps

1. Open Android Studio.
2. Create a new project using **Empty Views Activity**.
3. Name the project TableLayout.
4. Open activity_main.xml.
5. Replace the existing XML with the following code.
6. Run the application.
7. Enter the registration details and test the form.

PRACTICAL 4(a): ScrollView — Student Registration Form

Aim: Develop an Android application using ScrollView to display long, scrollable content.

Project Name: ScrollView

Main File: activity_main.xml

Steps

1. Open Android Studio.
2. Create a new project using Empty Views Activity.
3. Enter the project name ScrollView.
4. Select Java as the programming language.
5. Open activity_main.xml.
6. Replace the existing XML with the code below.
7. Run the application on an Android emulator.
8. Scroll down to view the complete registration form.

PRACTICAL 4(b): AbsoluteLayout — Fixed Positioning

Aim: Develop an Android application using AbsoluteLayout to demonstrate fixed positioning.

Project Name: AbsoluteLayout

Main File: activity_main.xml

Steps

1. Open Android Studio.
2. Create a new project using **Empty Views Activity**.
3. Enter the project name AbsoluteLayout.
4. Select Java as the programming language.
5. Open activity_main.xml.
6. Replace the existing XML with the code below.
7. Run the application on an Android emulator.

PRACTICAL 4(c): FrameLayout — Overlay UI Elements

Aim: Develop an Android application using FrameLayout to overlay UI elements.

Project Name: FrameLayout

Main File: activity_main.xml

Steps

1. Open Android Studio.
2. Create a new project using **Empty Views Activity**.
3. Enter the project name FrameLayout.
4. Select Java as the programming language.
5. Open activity_main.xml.
6. Add an image named baldski to the res/drawable folder.
7. Replace the existing XML with the code below.
8. Run the application on an Android emulator.

PRACTICAL 5(a): ListView with ArrayAdapter

Aim: Develop an Android application using ListView with ArrayAdapter and implement item click handling.

Project Name: ListViewDemo

Steps

1. Open Android Studio and create an Empty Views Activity project.
2. Set the project name to ListViewDemo.
3. Select Java as the programming language.
4. Open activity_main.xml and paste the XML code below.
5. Open MainActivity.java and paste the Java code below.
6. Run the application.
7. Tap any programming language to display a Toast message.

PRACTICAL 5(b): GridView with Custom Adapter

Aim: Develop an Android application using GridView with Custom Adapter and image selection handling.

Project Name: GridViewDemo

Steps

1. Create an Empty Views Activity project named GridViewDemo.
2. Open activity_main.xml and add a heading and GridView.

3. Create a new layout resource file named grid_item.xml.
4. Create a Java class named CustomAdapter.java.
5. Add four images to app/src/main/res/drawable/, named image1, image2, image3 and image4.
6. Replace the code in MainActivity.java.
7. Run the application and tap an image to see the selection message.

PRACTICAL 5(c): RecyclerView

Aim: Develop an Android application implementing RecyclerView as a modern alternative to ListView.

Project Name: RecyclerViewDemo

Steps

1. Open Android Studio.
2. Create an Empty Views Activity project named RecyclerViewDemo.
3. Select Java as the programming language.
4. Open activity_main.xml and paste the first code.
5. Create a layout file named item_layout.xml.
6. Create a Java class named MyAdapter.java.
7. Replace the code in MainActivity.java.
8. Add the RecyclerView dependency if your project doesn't already have it.
9. Run the application.

PRACTICAL 6(a): AppBar (Toolbar) and Options Menu

Aim: Develop an Android application implementing AppBar (Toolbar) and Options Menu.

Project Name: ToolbarMenuDemo

Steps

1. Open Android Studio.
2. Create an Empty Views Activity project named ToolbarMenuDemo.
3. Select Java as the programming language.
4. Replace the code in activity_main.xml.

5. Create menu_main.xml inside app/src/main/res/menu/. Create the menu folder if it doesn't exist.
6. Replace the code in MainActivity.java.
7. Run the application and open the menu using the three-dot icon.

I'll give you the aim, project name, steps and code for each practical in a copy-paste-friendly format. I'll also fix a few errors in the PDF that could prevent your code from running.

PRACTICAL 6(a): AppBar (Toolbar) and Options Menu

Aim: Develop an Android application implementing AppBar (Toolbar) and Options Menu.

Project Name: ToolbarMenuDemo

Steps

1. Open Android Studio.
2. Create an Empty Views Activity project named ToolbarMenuDemo.
3. Select Java as the programming language.
4. Replace the code in activity_main.xml.
5. Create menu_main.xml inside app/src/main/res/menu/. Create the menu folder if it doesn't exist.
6. Replace the code in MainActivity.java.
7. Run the application and open the menu using the three-dot icon.

1. Filename: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>


<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   android:id="@+id/main"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:orientation="vertical">
```

```
    <androidx.appcompat.widget.Toolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="?attr/actionBarSize"
        android:background="?attr/colorPrimary"
        android:theme="?attr/actionBarTheme"
        app:title="Toolbar Demo"
        app:titleTextColor="@android:color/white"/>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="AppBar and Options Menu Example"
        android:textSize="20sp"
        android:padding="22dp"/>

</LinearLayout>
```

2. Filename: menu_main.xml

Location: app/src/main/res/menu/menu_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<menu
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <item
        android:id="@+id/menu_home"
```

```
        android:title="Home"
        app:showAsAction="never"/>

    <item
        android:id="@+id/menu_about"
        android:title="About"
        app:showAsAction="never"/>

    <item
        android:id="@+id/menu_logout"
        android:title="Logout"
        app:showAsAction="never"/>

    <item
        android:id="@+id/menu_settings"
        android:title="Settings"
        app:showAsAction="never"/>

</menu>
```

3. Filename: MainActivity.java

```
package com.example.toolbarmenudemo;

import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.Toast;

import androidx.annotation.NonNull;
```

```
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.Toolbar;

public class MainActivity extends AppCompatActivity {

    Toolbar toolbar;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }

    @Override
    public boolean onOptionsItemSelected(
            @NonNull MenuItem item) {

        int id = item.getItemId();
```

```
if (id == R.id.menu_home) {
    Toast.makeText(
            this,
            "Home Selected",
            Toast.LENGTH_SHORT
    ).show();

} else if (id == R.id.menu_settings) {
    Toast.makeText(
            this,
            "Settings Selected",
            Toast.LENGTH_SHORT
    ).show();

} else if (id == R.id.menu_about) {
    Toast.makeText(
            this,
            "About Selected",
            Toast.LENGTH_SHORT
    ).show();

} else if (id == R.id.menu_logout) {
    Toast.makeText(
            this,
            "Logout Selected",
            Toast.LENGTH_SHORT
    ).show();
```

```
        } else {
            return super.onOptionsItemSelected(item);
        }

        return true;
    }
}
```

Expected output

The application displays a Toolbar titled **Toolbar Demo**. The options menu contains Home, About, Logout and Settings. Selecting an option displays a Toast message.

Remember: The Toolbar is the top bar; the Options Menu contains the actions available from that bar.

PRACTICAL 6(b): AlertDialog and Custom DialogFragment

Aim: Develop an Android application demonstrating AlertDialog and custom DialogFragment.

Project Name: DialogDemo

Steps

1. Create an Empty Views Activity project named DialogDemo.
2. Replace activity_main.xml with the code below.
3. Create dialog_custom.xml in res/layout.
4. Create a Java class named CustomDialogFragment.java.
5. Replace MainActivity.java with the code below.
6. Run the application and test both buttons.

PRACTICAL 6(c): Context Menu and Popup Menu

Aim: Develop an Android application implementing contextual menus and popup menus.

Project Name: MenuDemo

Steps

1. Create an Empty Views Activity project named MenuDemo.

2. Replace activity_main.xml.
3. Create menu_options.xml inside res/menu.
4. Replace MainActivity.java.
5. Run the application.
6. Long-press the text to open the Context Menu.
7. Tap **Show Popup Menu** to open the Popup Menu.

PRACTICAL 7(a): Explicit and Implicit Intents

Aim: Develop an Android application implementing Explicit and Implicit Intents.

Project Name: IntentDemo

Steps

1. Create an Empty Views Activity project named IntentDemo.
2. Create activity_main.xml.
3. Create another layout file named activity_second.xml.
4. Replace MainActivity.java.
5. Create a Java class named SecondActivity.java.
6. Register SecondActivity in AndroidManifest.xml if Android Studio hasn't already done so.
7. Run the application and test both buttons.

PRACTICAL 7(b): Event Handling Using Listeners

Aim: Develop an Android application demonstrating Event Handling using Listeners (OnClick, OnLongClick, etc.).

Project Name: EventHandlingDemo

Steps

1. Create an **Empty Views Activity** project named EventHandlingDemo.
2. Open activity_main.xml.
3. Add two buttons: **Click Me** and **Long Press Me**.
4. Replace the code in MainActivity.java.
5. Run the application.

6. Click the first button and long-press the second button to test both listeners.

PRACTICAL 7(c): Using Adapters to Populate Spinner and ListView

Aim: Develop an Android application using Adapters to populate Spinner or ListView dynamically.

Project Name: AdapterDemo

Steps

1. Create an **Empty Views Activity** project named AdapterDemo.
2. Open activity_main.xml.
3. Add a Spinner and a ListView.
4. Replace the code in MainActivity.java.
5. Create an array containing city names.
6. Use ArrayAdapter to display the cities in both the Spinner and ListView.
7. Run the application and select or click different cities.

PRACTICAL 8(a): Started and Bound Services

Aim: Develop an Android application implementing Started and Bound Services.

Project Name: ServiceDemo

Steps

1. Create an **Empty Views Activity** project named ServiceDemo.
2. Create activity_main.xml.
3. Create a Java class named MyStartedService.java.
4. Create another Java class named MyBoundService.java.
5. Replace the code in MainActivity.java.
6. Register both services in AndroidManifest.xml.
7. Run the application and test the **Start Service**, **Stop Service**, and **Bind Service** buttons.

PRACTICAL 8(b): Notification Manager

Aim: Develop an Android application implementing Notification Manager.

Project Name: NotificationDemo

Steps

1. Create an **Empty Views Activity** project named NotificationDemo.
2. Open activity_main.xml and add a **Show Notification** button.
3. Replace the code in MainActivity.java.
4. Add notification permission to AndroidManifest.xml.
5. Run the application and allow notification permission if prompted.
6. Click **Show Notification** and check the notification panel.

PRACTICAL 8(c): Static and Dynamic Broadcast Receivers

Aim: Develop an Android application implementing Static and Dynamic Broadcast Receivers.

Project Name: BroadcastDemo

Steps

1. Create an **Empty Views Activity** project named BroadcastDemo.
2. Create activity_main.xml.
3. Create a Java class named MyReceiver.java.
4. Replace the code in MainActivity.java.
5. Register the receiver in AndroidManifest.xml for the static receiver.
6. Run the application.
7. Turn Airplane Mode on or off to test broadcast reception.

PRACTICAL 8(c): Static and Dynamic Broadcast Receivers

Aim: Develop an Android application implementing Static and Dynamic Broadcast Receivers.

Project Name: BroadcastDemo

Steps

1. Create an **Empty Views Activity** project named BroadcastDemo.
2. Create activity_main.xml.
3. Create a Java class named MyReceiver.java.
4. Replace the code in MainActivity.java.
5. Register the receiver in AndroidManifest.xml for the static receiver.
6. Run the application.
7. Turn Airplane Mode on or off to test broadcast reception.

PRACTICAL 9(b): SQLite Database with ListView

Aim: Develop an Android application integrating an SQLite database with a ListView to display, add, update and delete records.

Project Name: SQLiteCRUDDemo

Steps

1. Create an **Empty Views Activity** project named SQLiteCRUDDemo.
2. Create activity_main.xml.
3. Create a Java class named DBHelper.java.
4. Replace the code in MainActivity.java.
5. Run the application.
6. Enter a name and click **Insert**.
7. Click **View Data** to display the saved names in the ListView.

PRACTICAL 9(c): Fetch JSON Data from a Web API

Aim: Develop an Android application to fetch JSON data from a web API and parse and display it in the Android application.

Project Name: JSONDemo

Steps

1. Create an **Empty Views Activity** project named JSONDemo.
2. Open activity_main.xml.
3. Add a **Fetch JSON Data** button and a TextView.
4. Replace the code in MainActivity.java.
5. Add Internet permission to AndroidManifest.xml.
6. Run the application with an active internet connection.
7. Click **Fetch JSON Data** to retrieve and display the JSON data.

PRACTICAL 10(a): Threads and Handlers for Background Processing

Aim: Develop an Android application demonstrating Threads and Handlers for background processing.

Project Name: ThreadHandlerDemo

Steps

1. Create an **Empty Views Activity** project named ThreadHandlerDemo.
2. Open activity_main.xml.
3. Add a **Start Background Task** button and a TextView.
4. Replace the code in MainActivity.java.
5. Run the application.
6. Click **Start Background Task**.
7. Wait approximately 3 seconds for the result to appear.

PRACTICAL 10(b): Media APIs and Telephone APIs

Aim: Develop an Android application implementing Media APIs and Telephone APIs.

Project Name: MediaTelePhoneDemo

Steps

1. Create an **Empty Views Activity** project named MediaTelePhoneDemo.
2. Create activity_main.xml.
3. Add two buttons: **Play Music** and **Open Dialer**.
4. Add an audio file named music.mp3 inside the res/raw folder.
5. Replace the code in MainActivity.java.
6. Run the application.
7. Click **Play Music** to play the audio.
8. Click **Open Dialer** to open the phone dialer with a number filled in.

PRACTICAL 10(c): Runtime Permissions and Security Handling

Aim: Develop an Android application implementing Runtime Permission and security handling in AndroidManifest.xml.

Project Name: PermissionDemo

Steps

1. Create an **Empty Views Activity** project named PermissionDemo.
2. Open activity_main.xml.
3. Add a button named **Request Camera Permission**.
4. Replace the code in MainActivity.java.

5. Add camera permission to AndroidManifest.xml.
6. Run the application.
7. Click **Request Camera Permission**.
8. Select an option in the permission dialog.
9. Observe the Toast message indicating whether permission was granted or denied.